

Proposta de Escopo de Aulas — Modelos Kimi na Prática

Resumo do direcionamento

Curso de um único módulo com 8 aulas, totalmente focado em benchmarking prático entre modelos: Kimi, Claude Opus, GPT-4o e Gemini. Todas as aulas são construídas em torno de comparações mensuráveis de custo × resultado, executadas principalmente via Jupyter Notebooks e integradas a ferramentas como o Cursor.

O Kimi Code CLI pode ser mencionado como alternativa, mas o foco pedagógico é a API Kimi/Moonshot conectada a ferramentas de desenvolvimento.

Público-alvo

- Desenvolvedores e estudantes de programação que querem escolher modelos de IA com critérios de custo e qualidade.
- Pessoas interessadas em integrar LLMs via API e comparar fornecedores de forma prática.

Objetivo de aprendizagem

Ao final do curso, o aluno será capaz de:

1. Configurar a API da Moonshot AI (Kimi) em scripts Python e no Cursor.
2. Comparar modelos de diferentes fornecedores usando prompts padronizados.
3. Medir e interpretar **custo de tokens** versus **qualidade do resultado**.
4. Aplicar benchmarks em cenários reais: texto, código, multimodalidade e produtividade em IDE.
5. Tomar decisões baseadas em dados sobre qual modelo usar em cada cenário.

Metodologia de Benchmarking

O curso é construído em torno de comparações práticas e mensuráveis entre os modelos. O mesmo prompt, a mesma tarefa e as mesmas métricas serão aplicados a todos os providers, gerando dados objetivos para análise.

Modelos comparados

Moonshot AI (Kimi)	OpenAI-compatible	Kimi k1.5 / Kimi latest
Anthropic	Anthropic Messages	Claude Opus 4 / Claude Sonnet 4
OpenAI	OpenAI Chat Completions	GPT-4o / GPT-4o-mini
Google	Google GenAI	Gemini 2.5 Pro / Gemini 2.0 Flash

Métricas padronizadas de comparação

Tokens de entrada	Tamanho do prompt	Retornado pela API

Tokens de saída	Tamanho da resposta	Retornado pela API
Custo estimado	Custo financeiro da chamada	Preço por 1M tokens × tokens consumidos
Tempo de resposta	Latência da geração	`time` ao redor da chamada
Qualidade funcional	O código/resultado funciona?	Testes, execução, lint
Qualidade textual/visual	O resultado atende ao pedido?	Rubrica ou comparação com referência
Eficiência de custo	Resultado obtido por dólar gasto	Score combinado de qualidade / custo

Roteiro de comparação entre modelos

Aula 4	**Benchmark textual**	Resumo, explicação técnica ou geração de ideias
Aula 5	**Benchmark de código**	Geração de função/código e passagem em testes
Aula 6	**Benchmark multimodal**	Landing page gerada a partir de screenshot
Aula 7	**Benchmark de automação**	Geração de testes, documentação e refatoração
Aula 7	**Benchmark em IDE**	Produtividade no Cursor usando Kimi vs concorrente
Aula 8	**Benchmark estratégico**	Trade-offs de contexto longo, velocidade e custo

Aulas

Aula 1 — Introdução e configuração do ambiente

- Apresentação do ecossistema Kimi: modelos, API Moonshot e Kimi Code CLI.
- Visão geral dos concorrentes: Claude Opus, GPT-4o, Gemini.
- Criação de contas e obtenção de API Keys.
- Instalação de Python, Jupyter Notebook e bibliotecas.
- Organização segura de credenciais com variáveis de ambiente.
- **Entregável:** ambiente configurado com as 4 chaves e notebook base rodando.

Aula 2 — Conectando a API do Kimi

- Primeiras chamadas à API Moonshot (protocolo OpenAI-compatible).
- Configurando a API do Kimi no Cursor.
- Configurando múltiplos providers no Kimi Code CLI (opcional/complementar).
- Estrutura de mensagens: system, user, assistant.
- **Entregável:** Cursor respondendo via API do Kimi e script Python fazendo chamada.

Aula 3 — Metodologia e template de benchmark

- Padronização de prompts: templates reutilizáveis.
- Parâmetros: `temperature`, `max\_tokens`, `top\_p`.
- Coletando tokens, tempo e custo.
- Criando a calculadora de custo.
- Apresentação do `template\_benchmark.ipynb`.
- **Entregável:** template de benchmark funcional com as 4 APIs.

Aula 4 — Benchmark textual

- Mesmo prompt executado em Kimi, Claude Opus, GPT-4o e Gemini.
- Tarefas: resumo, explicação técnica ou geração de ideias.
- Comparação de qualidade, tempo e custo.
- Criação de tabela comparativa e visualização.
- ****Entregável:**** relatório de benchmark textual.

Aula 5 — Benchmark de geração de código

- Estratégias de prompt para programação.
- Geração da mesma função/código pelos 4 modelos.
- Avaliação automática: execução, testes e lint.
- Comparação de custo por código funcional entregue.
- ****Entregável:**** script de avaliação de código e relatório comparativo.

Aula 6 — Benchmark multimodal: landing page a partir de screenshot

- Enviando uma imagem como input para cada modelo.
- Prompt único para recriar uma landing page a partir de screenshot.
- Comparação de fidelidade visual e estrutura do código.
- Custo de geração de cada modelo.
- ****Entregável:**** landing pages geradas e análise de custo × fidelidade.

Aula 7 — Benchmark de automação e produtividade em IDE

- Geração de testes unitários, documentação e refatoração via API.
- Comparando modelos em tarefas de manutenção de código.
- Desenvolvimento do mesmo pequeno projeto no Cursor com Kimi e concorrente.
- Registro de produtividade, qualidade e custo.
- ****Entregável:**** scripts de automação e dois protótipos comparados.

Aula 8 — Benchmark estratégico e conclusão

- Análise de trade-offs: contexto longo, velocidade, custo, multimodalidade.
- Revisão comparativa dos benchmarks realizados no curso.
- Publicação rápida dos projetos com GitHub Pages.
- Diagrama de decisão para escolha de modelo.
- Discussão sobre limitações e tendências.
- ****Entregável:**** fluxograma de escolha de modelo e publicação do projeto final.

Formato das aulas práticas

- ****Jupyter Notebooks (`.ipynb`)**** como principal ferramenta de ensino.
- Cada notebook de benchmark contém:
 - Objetivo da aula
 - Setup de credenciais e providers
 - Prompt padronizado
 - Execução com cada modelo

- Coleta de métricas (tokens, custo, tempo)
 - Avaliação/grade da resposta
 - Conclusão orientada a dados
 - **Notebook-base:** `template_benchmark.ipynb` serve de estrutura reutilizável para as aulas 4, 5, 6 e 7.
-

Observações importantes

- O **Kimi Code CLI** pode ser mostrado como alternativa de uso, mas o curso não depende dele.
 - O argumento central do curso é: **mesmo resultado com menor custo**, sustentado por dados coletados nos notebooks.
 - Todos os benchmarks devem ser reproduzíveis pelo aluno, com prompts e métricas documentados.
-

Próximos passos recomendados

1. Definir a duração total do curso e carga horária por aula.
2. Criar o `template_benchmark.ipynb` com as 4 APIs configuradas.
3. Validar preços atualizados das APIs (Kimi, Anthropic, OpenAI, Google) para cálculo de custo.
4. Escolher o screenshot de referência para a Aula 6.
5. Gravar um aula-piloto (Aula 4 ou Aula 5) para testar o formato.